

✓ REFINED INTERVIEW QUESTION LIST (High Expected Value)

◆ 1) C Language & Memory Internals

These are *expected* in any systems + ADA interview.

1. **Difference between pointer and array**
 - memory, sizeof, decay, mutability
2. **Explain array decay**
 - when it happens, when it doesn't
 - `sizeof`, `&arr`, function parameters
3. **Difference between `int *p[10]` and `int (*p)[10]`**
 - array of pointers vs pointer to array
4. **What is undefined behavior? Give 3 examples and why compilers exploit it.**
5. **Why is returning the address of a local variable dangerous?**
 - memory lifetime + stack frame
6. **What is pointer arithmetic?**
 - how pointer increments based on base type

Explain this expression:

```
char *c[] = {"one", "two", "three"};
printf("%s", c[1]+2);
```

7.

2) Dynamic Memory & Storage Classes

1. Explain **malloc**, **calloc**, **realloc**, and **free**.
 - differences + when to use
 2. Where in memory does **malloc** allocate?
 - heap organization
 3. Difference between stack and heap.
 - growth direction, lifetime, fragmentation
 4. What is static variable? What if you declare static inside function vs outside?
 5. What is **extern** and how does it work with multiple files?
 6. Explain storage classes in C.
-

3) Operating Systems Core

1. Describe process memory layout
 - text, data, bss, heap, stack
2. What is a stack frame? What is stored in it?
 - FP, PC, local vars, args
3. Stack vs Heap — key differences
 - performance, fragmentation, size
4. Stack pointer vs frame pointer
 - roles + interview way to answer

5. User stack vs kernel stack

- when each is used

6. What happens when a function call executes (assembly view)?

4) Linux / System Behavior

1. Describe process creation in Linux (fork + exec).

2. What is context switching?

3. How does `printf` actually work under the hood?

- buffer, write syscall

4. What happens when kernel handles a system call?

- trap, mode switch, stack switch

5. Difference between process and thread

- address space, stacks, scheduler
-



5) DSA — Time & Space Complexity

1. Explain $O(n^2)$, $O(n \log n)$, $O(\log n)$, $O(n^3)$

- with nested loops

2. Why binary search is $O(\log n)$?

3. Difference between array and linked list

- access time, memory, insertion cost

4. **When are hash tables better than trees?**
 5. **What is space complexity?**
 - auxiliary vs total space
 6. **Time complexity of common operations:**
 - push/pop in stack/queue
 - inserting in linked list
 - searching in binary tree
-

6) Recursion & Stacks

1. **Explain recursion stack growth (example factorial / fib).**
 2. **What is tail recursion?**
 3. **Memory overhead in recursion vs iteration**
 4. **Why recursion can cause stack overflow?**
 5. **Visualize call stack with activation records**
-

7) Interview Tricky / Edge Cases

(These often stump even great candidates)

What does this print and why?

```
int x = 5;  
printf("%d %d %d", x++, x++, x++);
```

- 1.

Explain this declaration:

```
int (*f())[10];
```

2.

What is the output here?

```
char s[] = "abcd";  
char *p = s;  
printf("%c %c", *p, *(p+1));
```

3.

4. **What happens if you free memory twice?**

5. **What is segmentation fault and why does it occur?**

8) System Design / Higher Thinking Questions

(Great for senior interviewers)

1. **How would you design a simple memory allocator?**

- free list, block splitting

2. **How does virtual memory help programs?**

- paging, TLB

3. **What is page fault?**

4. **How would you handle large data computation with limited memory?**

5. **Why do OS use caching?**

9) Work Experience Anchors

For each project on your resume:

1. **What was the problem you solved?**
2. **Why your chosen approach?**
3. **Alternatives you considered**
4. **What would you improve if given more time?**
5. **Constraints / bottlenecks you faced**
6. **Real-world edge cases you handled**

10) Linear Algebra (VERY IMPORTANT for ML interviews)

These are **most expected**, not advanced math proofs.

Core Concepts

1. **What is a vector? What is a matrix?**
 - geometric + data view
 2. **What does matrix multiplication represent?**
 - transformation / feature mixing
 3. **Why is matrix multiplication not commutative?**
 4. **What is rank of a matrix?**
 - relation to linear independence
 5. **What is determinant? What does $\det = 0$ mean?**
-

Eigen Concepts (VERY COMMON)

6. **What are eigenvalues and eigenvectors?**
 - intuition > formula
 7. **Why are eigenvectors important in ML?**
 - PCA, dimensionality reduction
 8. **What happens if eigenvalue is 0?**
 9. **Difference between diagonalizable and non-diagonalizable matrix (intuitive).**
-

Interview Traps

10. **What is inverse of a matrix? When does it exist?**
 11. **Why is pseudo-inverse used instead of inverse in ML?**
 12. **What is orthogonality? Why is it useful?**
-



11) Probability & Statistics (VERY HIGH ROI)

These are *almost guaranteed* if ML is on resume.

Basics

1. **What is probability?**
 - frequentist vs intuitive
2. **Conditional probability**
 - intuition + formula

3. Bayes' theorem

- interpretation, not rote formula

4. Independence vs mutual exclusivity

Random Variables

5. What is a random variable?

- discrete vs continuous

6. Expectation & variance

- meaning, not formula

7. Why variance is squared deviation?

Distributions (Very Important)

8. Gaussian distribution

- why it appears everywhere

9. Bernoulli, Binomial, Poisson

- where each is used

10. Law of Large Numbers

11. Central Limit Theorem

- *interview favorite*
-

Interview Traps

12. **Difference between PDF and CDF**
 13. **What does covariance measure?**
 14. **Correlation vs causation**
 15. **Why normal distribution is important in ML?**
-

12) Basic Machine Learning (SYSTEM + MATH VIEW)

They won't ask fancy models — they'll test **fundamentals**.

ML Foundations

1. **What is Machine Learning?**
 2. **Supervised vs Unsupervised learning**
 3. **Regression vs Classification**
-

Model Understanding

4. **What is loss function? Why do we minimize it?**
 5. **Difference between MSE and MAE**
 6. **What is gradient descent?**
 - geometric intuition
 7. **Why learning rate matters?**
 8. **What happens if learning rate is too large/small?**
-

Overfitting & Generalization

9. What is overfitting?
 10. Bias vs variance tradeoff
 11. How to reduce overfitting?
 - L1/L2, dropout, data
-

Linear Models (VERY COMMON)

12. Linear regression assumptions
 13. What happens if features are correlated?
 14. Why normalization is important?
-

Interview Traps

15. Why sigmoid for logistic regression?
 16. What is vanishing gradient?
 17. Why deep networks are hard to train?
-

13) ML + Systems Bridge Questions (GOLD)

These **differentiate you** from pure ML candidates.

1. Why is ML training memory intensive?
2. Why GPU is preferred over CPU for ML?

3. **What limits batch size?**
 4. **Why float32 instead of float64 in DL?**
 5. **How does cache/memory bandwidth affect ML performance?**
-

14) Resume-Based ML Questions (YOU MUST PREPARE)

For **each ML project**, be ready to answer:

1. **What was the input?**
 2. **What preprocessing was done?**
 3. **Why that model?**
 4. **What was the loss function?**
 5. **What were the failure cases?**
 6. **How would you deploy it efficiently?**
 7. **What happens if data distribution changes?**
-

15) Rapid-Fire Mixed Questions (Last-Round Style)

1. Why PCA works?
2. Why normalization helps gradient descent?
3. Why stack is faster than heap?
4. Why recursion uses more memory?
5. Why floating point is not associative?

6. Why ML models fail in production?
7. Why large models generalize better sometimes?